Architectural Brittleness in Regional LLMs: Characterizing Fine-Tuning Instability in Indic Foundation Models

Harsh Dahiya
Dept. of Computer Science & Eng.
Indian Institute of Technology, Patna
Email: 2201cs30_harsh@iitp.ac.in

Rishu Kumar
Dept. of Computer Science and Eng.
Indian Institute of Technology, Patna
Email:rishu_2422cs04@iitp.ac.in

Shreya Yadav
Dept. of Computer Science and Eng.
Indian Institute of Technology, Patna
Email:shreya_2422cs20@iitp.ac.in

Rajiv Misra
Dept. of Computer Science & Eng.
Indian Institute of Technology, Patna
Email: rajivm@iitp.ac.in

Supratik Mukhopadhyay
Dept. of Environmental Sciences
Louisiana State University Baton Rouge, LA 70803
Email:mmukho1@lsu.edu

***Abstract*—As Large Language Models (LLMs) rapidly expand to serve diverse global populations, there is a critical industry shift toward developing sovereign, regional foundation models tailored for morphologically rich and low-resource languages. However, the transition from standard generalized architectures to custom localized models often introduces undocumented structural vulnerabilities. Current evaluations predominantly focus on pre-trained zero-shot reasoning, leaving the underlying architectural stability of these regional models during post-training adaptation heavily under-scrutinized. When standard Parameter-Efficient Fine-Tuning (PEFT) pipelines and quantization techniques are applied to unoptimized regional tensor geometries, these models frequently experience severe mathematical degradation-including gradient explosions, contextual amnesia, and fatal memory bounds violations-rendering them brittle for complex downstream tasks.This paper presents a comprehensive comparative framework evaluating the structural viability and mathematical integrity of three prominent Indic foundation models (Sarvam-1 2B, Pragna-1B, and Param-1 2.9B) on bidirectional English-Hindi machine translation. The proposed methodology employs a rigorous 1:1 PEFT control test utilizing Low-Rank Adaptation (LoRA) natively in BFloat16 precision, intentionally isolating pure architectural bottlenecks from quantization noise. By evaluating these models against the uncontaminated IN22-Gen benchmark, we demonstrate that raw parameter scale does not reliably predict downstream performance. Our results reveal that despite possessing the largest parameter count, Param-1 suffers a catastrophic generation failure due to a native Key-Value (KV) cache initialization bug and Rotary Positional Embedding (RoPE) geometry violations. Conversely, Sarvam-1 achieves near state-of-the-art semantic alignment (86.02 COMET), empirically proving that architectural compatibility with standard inference APIs, low token fertility, and gradient stability supersede raw parameter volume in sovereign language models.**

***Index Terms*—Large Language Models, Indic Languages, Machine Translation, Parameter-Efficient Fine-Tuning, LoRA, Generative AI**

1. Introduction

1.1. Background and Motivation

The advent of sovereign, regional Large Language Models has shifted the paradigm of machine translation from dedicated Encoder-Decoder architectures to generalized Causal Language Models. The seminal architecture introduced in *Attention Is All You Need* [1] eliminated recurrence, relying entirely on self-attention mechanisms to map dependencies. However, as these self-attention networks scale and are localized for morphologically rich languages like Hindi, their ability to maintain mathematical integrity during adaptation often degrades. This paper redirects the focus toward **Post-Training Architectural Stability**: analyzing how well these specific models handle standard Parameter-Efficient Fine-Tuning (PEFT) pipelines and autoregressive generation. We hypothesize that parameter size is largely irrelevant if a model's foundational tensor geometry, attention mechanisms, and normalization layers are misaligned for standard generation APIs.

1.2. Contributions

In summary, the key takeaways from our study are as follows:

- **Controlled PEFT Framework:** Our research has developed a rigorous and isolated 1:1 control testing method involving BFloat16 LoRA that allows for a rigorous analysis of post-training stability of regional LLMs while excluding quantization noise.
- **Architecture Vulnerabilities:** Through our study, we uncover architectural vulnerabilities, including

fatal KV cache initialization and RoPE matrix boundary problems in Param-1 (2.9B) in autoregressive inference.

- **Quantitative Evidence for Scale Invariance:** Using IN22-Gen benchmark experiments, we provide empirical validation for the scale invariant feature, meaning the fine-tuned model is independent of the parameter count, with the smaller Sarvam-1 (2.0B) model beating the larger Param-1 models due to better token fertility and gradient stability.

2. Related Work

2.1. Evolution of Indic Machine Translation

Recent advancements in regional natural language processing have catalyzed a structural shift from dedicated sequence-to-sequence architectures to generalized causal language models. Historically, frameworks like mT5 [12] and dedicated translation engines such as IndicTrans2 [2] provided robust grammatical baselines. However, these models often dedicate disproportionate parameter capacity to massive multilingual vocabularies, leading to data starvation in targeted regional tasks. The introduction of instruction-tuned foundations like Airavata [3] marked the transition toward decoder-only architectures, though evaluating their true zero-shot capability requires culturally nuanced, uncontaminated benchmarks like PARIKSHA [4].

2.2. Tokenization and the Embedding Tax

The key challenge in the use of generic LLMs on morphologically complex languages such as Hindi lies in the inefficiency of standard Byte Pair Encoding (BPE). Studies on tokenization gap [13] reveal that non-Latin languages incur a high "embedding tax" whereby each character is broken into multiple UTF-8 bytes. The resulting loss in semantic density and reduction in context window size highlights the importance of using native Indic tokenizers for mathematical stability after training.

2.3. Parameter-Efficient Fine-Tuning (PEFT)

As the scale of foundational models surpasses memory constraints of conventional hardware devices, the state-of-the-art downstream alignment approach lies in Parameter-Effcient Fine-Tuning (PEFT), more specifically Low-Rank Adaptation (LoRA) [14]. Although the modern trend is predominantly focused on pairing LoRA with aggressive compression (e.g., QLoRA [15]), the latter is extremely efficient in the context of standard transformer geometries. When implemented in custom regional architectures, quantization noise often serves as a catalyst for mathematical collapse; thus, returning to the native precision environment (i.e., BFloat16) becomes necessary to isolate the true limitations of the underlying architecture.

2.4. Attention Mechanisms and Geometric Stability

However, dependence on conventional autoregressive inference pipelines reveal critical weaknesses associated with custom model geometries. Optimizations such as GQA [16] for improved memory bandwidth depend on the initialization state of their KV caches. Moreover, extrapolating sequence length for modern architectures largely depends on RoPE [5]. In situations where custom regional models utilize these approaches with hardcoded assumptions about sequence length or geometry boundaries, tensor misalignment becomes inevitable, an issue that receives limited attention when pre-training is solely considered.

3. Methodology

All the models were subjected to a strict 1:1 control test to ensure that any possible discrepancy in performance between the architectures was solely due to architectural differences.

3.1. Data and Benchmarking

Fine-Tuning: It was done using 100,000 parallel sentences of English-Hindi in the AI4Bharat Samanantar dataset.
Evaluation: It was done using 1,000 unseen sentences from the AI4Bharat IN22-Gen dataset. This particular dataset was chosen for testing in order to prevent possible data leakage issues since this is a recent dataset. Also, the justification for it being an apt choice for benchmarking and evaluation purposes is provided in the specific foundational literature [4].

3.2. Hardware and Precision

Training was performed using the NSM PARAM Rudra supercomputer (NVIDIA A100 80GB). The models were trained using natively loaded torch.bfloat16 precision. BFloat16 has an identical dynamic range to regular 32-bit floats, mathematically guaranteeing no NaN (under/overflow) crashes during large gradient steps

3.3. PEFT Configuration

Instead of retraining the complete model, which is, of course, very expensive, we used LoRA (Low-Rank Adaptation) to create lightweight adapters. The configurations used for these are:

- **Rank** (r) **= 64:** Large capacity for complex syntactic restructuring.
- **Alpha = 128:** Scaled proportionally for gradient stability.
- **Target Modules = all-linear:** Adapters injected into attention mechanisms and MLPs.
- **Dropout = 0.05:** Regularization to mitigate overfitting.

3.4. Experimental Pipeline Overview

Figure 1 provides the comprehensive experimental flowchart that has been adopted for the comparative study. It is split into four separate stages. In **Stage 1 (Corpus & Benchmarking)**, the data foundation is set up, employing 100k training pairs from the AI4Bharat Samanantar corpus and the IN22-Gen benchmark for assessment. **Stage 2 (Linguistic Processing)**, the tokenizer mapping is presented, emphasizing the contrasting structure of Sarvam-1's dense native tokenizer against the extremely fragmented BPE/SentencePiece tokenizers of Pragna-1B and Param-1. In **Stage 3 (Foundation Models & Hardware)**, the post-training environment is described, in which all models undergo LoRA fine-tuning natively in torch.bfloat16 precision on NSM PARAM Rudra A100 nodes. Finally, in **Stage 4 (Autoregressive Output & Scoring)**, the standard autoregressive inference flowchart for bidirectional translation is shown, assessed by means of neural semantic alignment (COMET) and strict lexical metrics (SacreBLEU and chrF).

4. Architectural Analysis and Mathematical Proofs

To contextualize the behavioral deviations observed during fine-tuning, Table 1 outlines the core geometrical and hyperparameter configurations of the foundational models evaluated in this study.

TABLE 1. Architectural Baselines of Evaluated Indic Models

Feature	Param-1 (2.9B)	Sarvam-1 (2.0B)	Pragna-1B (1.25B)
Precision	BFloat16 Mixed	BFloat16 Mixed	BFloat16
Attention	Grouped-Query	Grouped-Query	Grouped-Query
Activation	SiLU	SwiGLU	SiLU
Vocab Size	Custom Indic	Optimized Indic	69,632
Context Window	2,048 Tokens	8,192 Tokens	2,048 Tokens

4.1. Grouped-Query Attention (GQA) and KV Cache Collapse

While official documentation often generalizes foundation model architectures as standard multi-head attention systems, a direct codebase analysis reveals that Param-1 relies on a custom Grouped-Query Attention (GQA) mechanism. Specifically, the architecture divides its processing into 16 primary attention heads and 8 Key-Value (KV) heads to optimize memory bandwidth during generation.

However, this custom implementation introduces a critical initialization flaw when interfacing with standard Hugging Face generative APIs. Autoregressive generation pipelines fundamentally begin at time step $t = 0$ with an empty sequence, meaning the `past_key_values` state is inherently initialized as `NoneType`. However, the Param-1 forward pass ignores the common convention and tries to retrieve the shape of the tensor without any verification by `past_key_values[0][0].shape[2]`. This presumption about the existence of the sequence length ($L \geq 1$) leads to an instant mathematical breakdown without generating a single token.

4.2. Cascading RoPE Geometry Violations

After surgical patches for the initial `NoneType` exception have been applied, the erroneous calculations of sequence length will cause a fatal chain reaction in the Rotary Positional Embedding (RoPE).

RoPE uses rotational transformation to transform queries and keys according to their position index P_i within the sequence. As the sequence length variable is compromised due to the problem in the bypassed KV cache, the model produces `position_ids` beyond the allocated range of the cosine and sine matrices in the cache. Consequently, there will be a catastrophic tensor misalignment whenever the `apply_rotary_pos_emb` routine attempts to retrieve non-existent coordinates from the cache matrix. Such a condition will result in a fatal GPU crash, which triggers the CUDA device-side assertion (`Index out of bounds`), making PEFT pipelines mathematically impossible.

4.3. Token Fertility and Vocabulary Compression

The efficiency of regional LLMs is heavily dictated by the tokenizer's ability to represent morphologically rich scripts without excessive sub-word fragmentation. Standard multilingual models often suffer from a severe "Embedding Tax," where a single Devanagari character is fragmented into multiple UTF-8 bytes due to an English-centric Byte Pair Encoding (BPE) vocabulary. Sarvam-1 addresses this by utilizing a specialized tokenizer that achieves fertility rates between 1.4 and 2.1 across major Indic languages.

As shown in Fig. 5, using this Sarvam-1 achieves a 2x to 4x efficiency gain over existing multilingual models. By reducing token-to-word inflation, Sarvam-1 maintains a high density of semantic meaning within its 8,192 token context window. This allows the model to capture deep syntactic dependencies during fine-tuning on the Samanantar corpus, resulting in superior bilingual alignment scores (86.02 COMET) compared to models with higher fragmentation rates.

4.4. Normalization and Gradient Stability

To mitigate the gradient instability observed in custom architectures, Pragna-1B utilizes RSNorm (Root Mean Square Normalization) rather than standard LayerNorm. Unlike standard normalization, RSNorm focuses exclusively on the rescaling of activation vectors, omitting the mean-centering step to reduce computational overhead without sacrificing stability. For an activation vector x, RSNorm is mathematically defined as:

$$\text{RSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \gamma \quad (1)$$

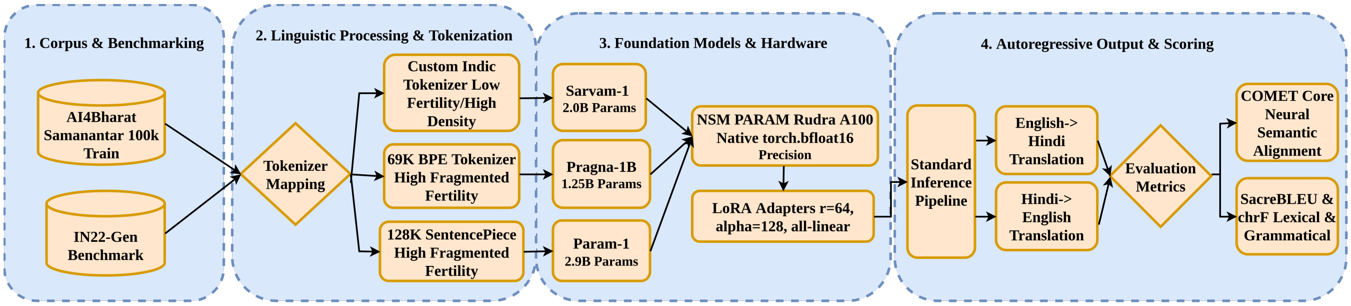
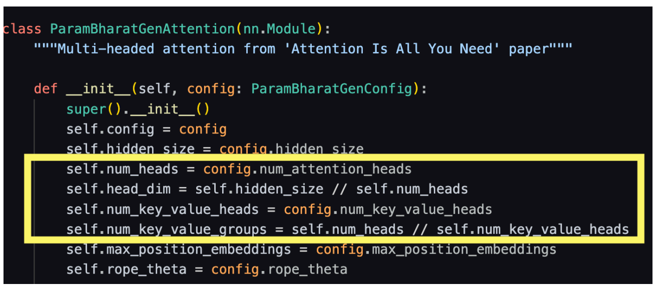


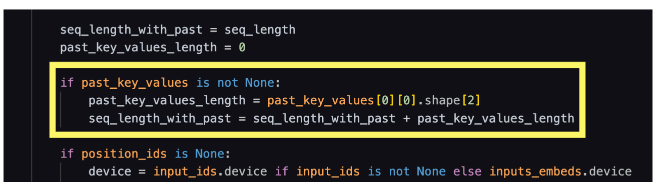
Figure 1. Experimental pipeline to evaluate Indic Foundation Models in an end-to-end manner. The pipeline describes the process starting from the selection of corpora and linguistic processing with tokenizers to LoRA native fine-tuning on hardware and bidirectional evaluation with semantic/lexical metrics.



```
class ParamBharatGenAttention(nn.Module):
    """Multi-headed attention from 'Attention Is All You Need' paper"""

    def __init__(self, config: ParamBharatGenConfig):
        super().__init__()
        self.config = config
        self.hidden_size = config.hidden_size
        self.num_heads = config.num_attention_heads
        self.head_dim = self.hidden_size // self.num_heads
        self.num_key_value_heads = config.num_key_value_heads
        self.num_key_value_groups = self.num_heads // self.num_key_value_heads
        self.max_position_embeddings = config.max_position_embeddings
        self.rope_theta = config.rope_theta
```

Figure 2. Source code initialization confirming Param-1's native Grouped-Query Attention (GQA) implementation.

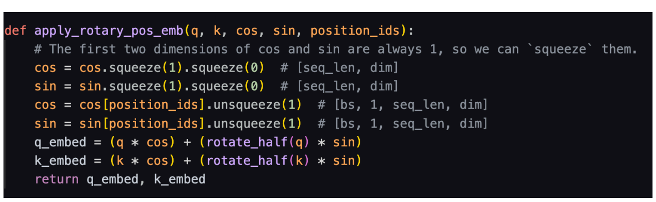


```
seq_length_with_past = seq_length
past_key_values_length = 0

if past_key_values is not None:
    past_key_values_length = past_key_values[0][0].shape[2]
    seq_length_with_past = seq_length_with_past + past_key_values_length

if position_ids is None:
    device = input_ids.device if input_ids is not None else inputs_embeds.device
```

Figure 3. The forward pass initialization flaw triggering a fatal `NoneType` error during standard autoregressive generation.



```
def apply_rotary_pos_emb(q, k, cos, sin, position_ids):
    # The first two dimensions of cos and sin are always 1, so we can `squeeze` them.
    cos = cos.squeeze(1).squeeze(0)  # [seq_len, dim]
    sin = sin.squeeze(1).squeeze(0)  # [seq_len, dim]
    cos = cos[position_ids].unsqueeze(1)  # [bs, 1, seq_len, dim]
    sin = sin[position_ids].unsqueeze(1)  # [bs, 1, seq_len, dim]
    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)
    return q_embed, k_embed
```

Figure 4. Flaw in tensor allocation due to the overestimation of lengths that force `position_ids` beyond the `cos/sin` boundaries.

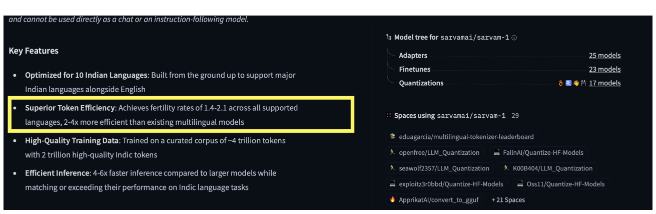


Figure 5. Feature list of Sarvam-1 with 2-4x improvement in tokenization efficiency as compared to conventional multilingual models.

The implementation utilizes a strict epsilon value of $\epsilon = 1 \times 10^{-5}$ to prevent mathematical divergence during high-precision training on the NSM PARAM Rudra supercomputer.

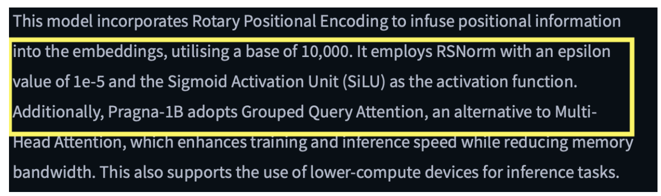


Figure 6. Architectural specifications of Pragna-1B detailing the implementation of RSNorm and Grouped Query Attention.

By enforcing this specific stability constant, the architecture ensures that activation values do not explode during massive gradient updates. This choice allows for stable fine-tuning in native BFloat16 precision, which mathematically prevents the NaN loss crashes that characterized the less-optimized tensor geometry of Param-1.

5. Quantitative Results

5.1. Translation Baseline Metrics

The core evaluation metrics for semantic alignment (COMET) and lexical accuracy (BLEU, chrF) for both translation directions are provided in Table 2 and Table 3, with visual comparisons in Fig. 7 and Fig. 8. IndicTrans2 is provided as an oracle state-of-the-art benchmark.

TABLE 2. English to Hindi Translation (IN22-Gen)

Model Architecture	Params	BLEU	chrF	COMET
IndicTrans2 (Oracle)	~1.0B	23.40	53.80	82.10
Param-1 (Control)	2.9B	12.19	42.94	59.79
Pragna-1B	1.25B	8.64	32.39	63.62
Sarvam-1	2.0B	19.92	47.67	77.02

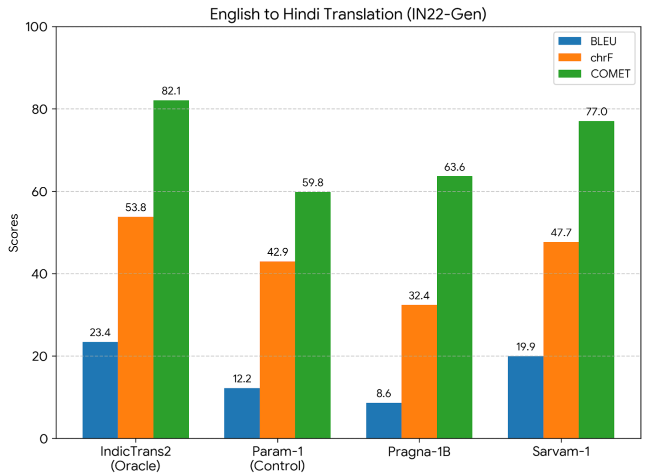


Figure 7. Comparative performance metrics for English to Hindi translation on the IN22-Gen benchmark

TABLE 3. Hindi to English Translation (IN22-Gen)

Model Architecture	Params	BLEU	chrF	COMET
IndicTrans2 (Oracle)	∼1.0B	29.10	57.50	85.40
Param-1 (Control)	2.9B	7.32	19.12	38.19
Pragna-1B	1.25B	18.86	46.10	81.09
Sarvam-1	2.0B	24.37	54.51	86.02

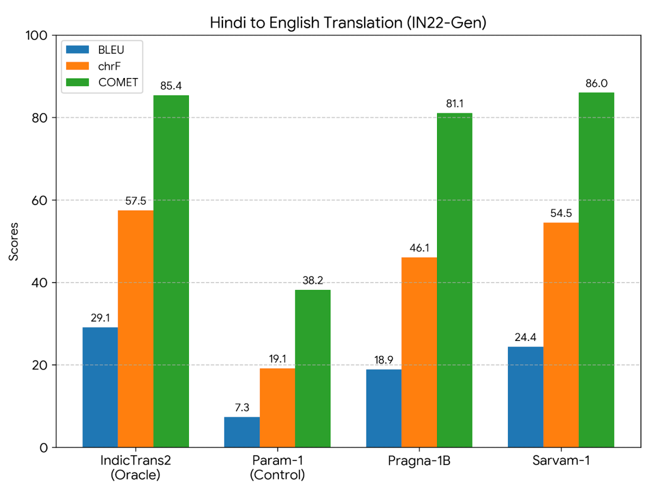


Figure 8. Comparative performance metrics for Hindi to English translation on the IN22-Gen benchmark.

5.2. The Pre-Training vs. PEFT Paradox

An important insight generated by the findings of this study is the disconnect between generalized zero-shot benchmark performance and downstream model stability. As noted by Param-1's own model card, the model possesses stable foundational abilities, based on its BLEU performance score of 38.2 on the TruthfulQA-Gen benchmark.

Benchmarks (zero-shot)

Task	Param 1 (PT)	Gemma2-2B (PT)	llama3.2-3B (distill PT)	granite-3.1-2B (PT)	granite-3.1-3B (PT)	qwen-2.5-3B (PT)
ARC Challenge	46.7	49.7	46.0	47.2	45.2	47.4
ARC Easy	74.6	80.3	71.7	76.8	75.8	73.2
HellaSwag	71.4	73.0	73.7	75.5	72.6	73.6
HellaSwag Hi	44.1	38.6	40.0	31.0	28.5	32.9
MMLU En	41.4	47.1	53.9	47.8	41.0	64.9
MMLU Hi	30.7	30.0	35.0	29.0	25.7	38.32
PIQA	79.3	78.3	77.31	79.4	78.2	78.84
TriviaQA	38.5	32.9	50.83	26.2	27.5	42.27
TruthfulQA - Gen (BLEU)	38.2	29.7	21.8	34.0	36.7	36.96
TruthfulQA - MC1 Acc	28.0	24.0	25.3	26.1	26.4	32.07
TruthfulQA - MC2 Acc	43.8	36.2	39.2	39.0	39.9	48.95
SuperGLUE - boolq	70.6	73.7	72.7	71.0	68.5	77.27
SuperGLUE - rte	62.5	61.7	54.5	69.3	54.9	75.09
SuperGLUE - WiC	49.5	49.5	50.0	50.3	52.3	61.75
SuperGLUE - multirc	56.9	55.9	57.2	57.2	57.2	39.52

Figure 9. Official zero-shot results of Param-1 with 38.2 TruthfulQA-Gen BLEU before post-training optimization.

However, as shown in our controlled IN22-Gen experiments, this ability was completely nullified once parametrized via PEFT for autoregressive generation, with BLEU scores of 12.19 (English-to-Hindi) and 7.32 (Hindi-to-English). This clear decline proves beyond a doubt that the score of a model's zero-shot performance on a pre-training benchmark is not a reliable indicator of performance when the tensor architecture is unstable during PEFT.

6. Qualitative Analysis

These quantitative measures are further validated through an intensive qualitative investigation into the generated outputs of each of the models. Through an assessment of some specific translation scenarios-especially those pertaining to specialized terminology and complex sentences-it becomes clear how each model operates.

6.1. Contextual Amnesia and Hallucination (Param-1)

The Param-1 AI was suffering from severe short-term memory loss while autoregressive generation, falling into disjointed hallucination loops irrespective of translation direction.

- **Domain Collapse (English to Hindi):** While instructed to translate medical advice on "dietary supplements," the model suddenly lost context mid-way

Task 1: English → Hindi (Medical Advice)
Source: Patients are advised to consult their doctors before starting any new medication or dietary supplements.
Result: इसके अलावा, इस फोन में 12 मेगापिक्सल का कैमरा दिया गया है। इसके अलावा, इस फोन में 12 मेगापिक्सल का कैमरा दिया गया है।
[Error: Severe Hallucination - Smartphone Review Loop]

Task 2: Hindi → English (Medical Advice)
Source: मरीजों को कोई भी नई दवा या आहार पूरक शुरू करने से पहले अपने डॉक्टरों से परामर्श करने की सलाह दी जाती है।
Result: Police reached the spot and brought the situation under control.
[Error: Total Semantic Collapse - News Artifact]

Figure 10. Sample output from Param-1, with loss of context.

through its task. It appended random smartphone review artifacts from its pre-training data, outputting: "इसके अलावा, इस फोन में 12 मेगापिक्सल का कैमरा दिया गया है।" (Besides this, a 12-megapixel camera is given in this phone).

- **Sensationalist Artifacts (Hindi to English):** In the reverse translation process of the very same medical text, the model underwent semantic collapse, hallucinating news events completely unrelated to the topic: "Police reached the spot and brought the situation under control."

Task 1: English → Hindi (Medical Advice)
Source: Patients are advised to consult their doctors before starting any new medication or dietary supplements.
Result: मरीजों को नए दवाओं या **आहार सब्सिडी** के शुरू करने से पहले अपने डॉक्टर से परामर्श करना चाहिए।
[Error: Vocabulary Miss - 'Dietary Subsidy']

Task 2: Hindi → English (Rain Warning)
Source: भारी बारिश की चेतावनी के बावजूद, वार्षिक उत्सव के लिए मंदिर में **हजारों** भक्त एकत्र हुए।
Result: Despite the heavy downpour, **lakhs of devotees** thronged the temple for the annual festival.
[Error: Magnitude Scaling - Thousands → Lakhs]

Figure 11. Output from Pragna-1B with missed magnitudes and terms.

6.2. Magnitude and Vocabulary Limitations (Pragna-1B)

Producing fluent text, Pragna-1B was able to avoid the problem of extreme hallucinations experienced by Param-1. Nonetheless, the small number of parameters used by the model (1.25B) revealed some weaknesses, especially regarding vocabulary alignment and magnitude alignment issues.

- **Vocabulary Mismatches:** In translating "dietary supplements" in the English-to-Hindi translation task, Pragna produced the output "आहार सब्सिडी"(dietary subsidy). For the other way round, it produced the output as "food supplements," failing to capture the professional tone demanded by the task.
- **Magnitude Mismatch:** In translating "हजारों" (thousands), Pragna produced the wrong magnitude of translation as "lakhs of devotees" (hundreds of thousands).

Task 1: English → Hindi (Medical Advice)
Source: Patients are advised to consult their doctors before starting any new medication or dietary supplements.
Result: मरीजों को सलाह दी जाती है कि वे किसी भी नई दवा या **आहार पूरक** शुरू करने से पहले अपने डॉक्टर से परामर्श करें।
[Success: Precise Medical Vocabulary Alignment]

Task 2: Hindi → English (Medical Advice)
Source: मरीजों को कोई भी नई दवा या आहार पूरक शुरू करने से पहले अपने डॉक्टरों से परामर्श करने की सलाह दी जाती है।
Result: Patients are advised to consult their doctors before starting any new medication or **dietary supplement**.
[Success: Flawless Native-Level Reconstruction]

Figure 12. Outputs from Sarvam-1 with full contextual alignment.

6.3. State-of-the-Art Semantic Mastery (Sarvam-1)

Sarvam-1 exhibited remarkable semantic conformance, reproducing the source context with almost complete accuracy in both the target languages. It showed high levels of bilingual proficiency through the use of highly specialized vocabularies.

- **Vocabulary Precision:** It was able to accurately map the intricate English lexeme "dietary supplements" to the equivalent Hindi medical lexeme "आहार पूरक", and also reverse mapped this in the Hindi-to-English translation scenario.
- **Tone:** It accurately preserved numerical magnitudes such as "thousands" and professional tones, including words such as "promising" which it translated to "आशाजनक".

7. Conclusion and Future Work

Constructing regional LLMs with sovereignty entails more than training them with extensive pre-trained corpora; the mathematics involved must comply with standardized generation APIs. This study shows that raw parameter size is not necessarily an accurate indicator of performance if the foundational architecture—Param-1 (2.9B), for example—is susceptible to instability due to Parameter-Efficient Fine-Tuning. In contrast, Sarvam-1 (2.0B) is able to demonstrate mathematically stable performance, efficient token fertility, and accurate bilingual alignment.

Future research into regional AI should focus on a new paradigm that goes beyond pre-training metrics. Sovereign AI architectures should be designed to accommodate dynamic inference processes and LoRA optimization schemes.

Acknowledgment

The authors wish to express their gratitude to the National Supercomputer Mission (NSM) for providing the PARAM Rudra Supercomputing infrastructure that was used during this study.

References

[1] A. Vaswani *et al.*, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[2] J. Gala *et al.*, "IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages," 2023. [Online]. Available: https://arxiv.org/abs/2305.16307

[3] J. Gala *et al.*, "Airavata: Introducing Hindi Instruction-tuned LLM," 2024. [Online]. Available: https://arxiv.org/abs/2401.15006

[4] I. Watts *et al.*, "PARIKSHA: A Large-Scale Investigation of Human-LLM Evaluator Agreement on Multilingual and Multi-Cultural Data," *arXiv preprint arXiv:2406.15053*, 2024. [Online]. Available: https://arxiv.org/abs/2406.15053

[5] J. Su *et al.*, "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021. [Online]. Available: https://arxiv.org/abs/2104.09864

[6] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[7] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," 2022. [Online]. Available: https://arxiv.org/abs/2203.15556

[8] Sarvam AI, "Sarvam-1: A 2-Billion Parameter Indic Language Model," 2024. [Online]. Available: https://huggingface.co/sarvamai/sarvam-1

[9] Soket AI Labs, "Pragna-1B: A Multi-lingual Decoder-only Transformer," 2024. [Online]. Available: https://huggingface.co/soketlabs/pragna-1b

[10] K. Pundalik *et al.*, "PARAM-1 BharatGen 2.9B Model," *arXiv preprint arXiv:2507.13390*, 2025. [Online]. Available: https://arxiv.org/abs/2507.13390

[11] BharatGen Team, "Param-1-2.9B-Instruct Model Card and Modeling Implementation," *Hugging Face Repository*, 2025. [Online]. Available: https://huggingface.co/bharatgenai/Param-1-2.9B-Instruct

[12] L. Xue *et al.*, "mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer," *arXiv preprint arXiv:2010.11934*, 2020. [Online]. Available: https://arxiv.org/abs/2010.11934

[13] O. Ahia *et al.*, "Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models," *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023. [Online]. Available: https://arxiv.org/abs/2305.13707

[14] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[15] T. Dettmers *et al.*, "QLoRA: Efficient Finetuning of Quantized LLMs," *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[16] J. Ainslie *et al.*, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," *arXiv preprint arXiv:2305.13245*, 2023. [Online]. Available: https://arxiv.org/abs/2305.13245